iExtract iOS Album to Folder Conversion Tool

Basic Team Information:

- **(Names: Roles):**
  - Sam Daughtry: Developer
  - Noah Gregie: UI/UX
  - Kevin Gustafson: Director
  - Brendon Wong: QA Tester
- **GitHub Repo Link:** https://github.com/Gustakev/cs362-class-project
- **GitHub AI Image Description Feature Repo:** https://github.com/BrendxnW/image-captioning-cnn-lstm
- **Communication Rules:**
  - We will communicate primarily through a Microsoft Teams group chat and our OSU associated emails. We may establish other means of communication amongst ourselves, but that is optional.
  - In terms of timeliness, as long as someone responds to a message sent via Teams or email within a few hours, it will be considered acceptable. However, there may be some times when quicker communication is expected and/or necessary to reach deadlines on time. During those times, it is preferable that team members respond to messages from other members within at least an hour, so that we may accomplish our goals on time.

Project Milestone-3: Project Architecture and Design:

- **Software Architecture:** Layered Architecture
- **Major Components:**
  - CLI: The CLI, as part of the Presentation Layer, will accept user input, send that input to the Application Layer in a predictable manner, and the Application Layer will call (or import data from) one of the lower layers, the Domain Layer or the Data Layer, to make a request given the data it received from the CLI.
  - Textual UI: The Textual UI allows users to use a GUI within the terminal to do all the same things the aforementioned CLI can do but in a more intuitive manner.

    - Backup Locator & Validator: This will work by having a certain backup path handed down from the Presentation Layer to the Application Layer, which will first determine whether it could even be a real file path to begin with, but if so, will then proceed to sending it to the Data Layer, along with the correct SQL to execute, and then the Data Layer will open the correct database and execute the correct SQL using SQLite, and then it will return the SQLite response to the Application Layer, which will validate the response and create a persistent data structure representing the information from the databases, and then that will be returned to the Presentation Layer to render.
    - SQLite Query Component (Manifest.db + Photos.sqlite): This is the part of the Data Layer that executes the SQL query specified by the Application Layer on the specified file and returns the response data back to the Application Layer.
    - File Extraction Engine: This component is responsible for reconstructing an album/collection into a folder of media items. It simply receives an altered version of the object representing the backup, from the Application Layer, based on user choices in the Presentation Layer, along with another object from the Application Layer specifying what to create folders for, and then it creates the correct extraction based on the states of those objects.
    - Conversion Engine (HEIC to PNG, MOV to MP4, Live Photo to MP4): This component is responsible for interrupting the file extraction process whenever the file extraction engine attempts to copy a file that is marked as needing to be converted first, taking control of the process for a moment, converting the file, and then allowing the process to continue.
- **Interface(s) Between Components:**
    - CLI to Backup Locator & Validator:
        - The Presentation Layer passes a backup directory path string to BackupService.attempt_load_backup() in the Application Layer. The Backup Locator & Validator returns a BackupModelResult containing either a populated BackupModel object and an optional iCloud warning string on success, or an error string on failure.
    - Backup Locator & Validator to SQLite Query Component:

        - The backup_model_builder requests data from the SQLite Query Component by passing a database file path (Photos.sqlite or Manifest.db). The SQLite Query Component opens the database via SQLiteConnectionManager, runs the appropriate query via SQLExecutor, and returns a list of row dictionaries via RowMapper for the Application Layer to process.
    - Backup Locator & Validator to File Extraction Engine:
        - After constructing the BackupModel, the Application Layer passes it to ExportService.export_all() or ExportService.export_single_album(), along with a Blacklist object built by SettingsService and a conversion dictionary from ConversionService. The File Extraction Engine's run_extraction_engine() function consumes these inputs to perform the extraction.
    - File Extraction Engine to Conversion Engine:
        - When an asset requires format conversion, the File Extraction Engine calls convert_asset() in the Conversion Engine with an AssetToConvert object that bundles the asset and the active conversion type dictionary. The Conversion Engine returns a ConvertedAsset object; on success it includes the path to the converted temporary file.
    - File Extraction Engine to Presentation Layer:
        - The File Extraction Engine updates a shared progress object's percent attribute throughout the extraction. The Presentation Layer polls this value from a background thread to render a live progress bar. Upon completion, ExportService returns a (bool, str) tuple to the Presentation Layer indicating success or failure with a descriptive message.
- **Data Storage Explanation (High Level Schema or Organization):**
    - Our system does not maintain its own database. Instead, it queries two iOS backup databases (Photos.sqlite and Manifest.db) and uses the results to construct a BackupModel — a hierarchy of Pydantic objects representing the backup's device metadata, albums, and assets. This object is held in memory by BackupService for the duration of the session and is never written to disk.
- **Assumptions Underpinning Architecture Choice:**

    - Stability of iOS Backup Format: We make the assumption that Photos.sqlite and Manifest.db adhere to a stable schema that can be consistently queried during development.
    - Cross-Platform SQLite Support: We assume that SQLite operates uniformly on Linux, macOS, and Windows, allowing for a single Data Layer.
    - Primarily CLI Presentation: The system is primarily used via a command-line interface. A Textual TUI (textual_main_menu.py) has since been added as a beta alternative interface, but the Standard CLI remains the primary Presentation Layer.
    - In-Memory Backup Model: We are making the assumption that the user's photo library databases are sufficiently small to be fully rebuilt in the Application Layer's memory.
    - Availability of the File System Tools: We assume that the host system has the required file operations necessary to run the program.
    - User Permissions: We assume users have read access to the backup directory and write access to the destination directory.
    - I/O-Heavy Workflow: We assume the system's complexity lies primarily in I/O and workflow orchestration, making layered architecture an appropriate choice.
- **First Decision Pertaining to Architecture Choice (Layered Architecture Choice):**
    - Identify And Describe an Alternative:
        - Microkernel Design: A single microkernel that facilitates interactions between program components, which would all be plugins, and implements only the most basic operations.
    - Identify Pros Relative to Our Actual Choice:
        - It would make it easier for us to integrate plugins later on without needing to create a more complex plugin pattern that aligns with the Layered Architecture.
        - It would make it easier for the team to work on things simultaneously.
    - Identify Cons Relative to Our Actual Choice:
        - It would be more difficult to design a microkernel that respects multiple ways of communication with different plugins than it would be to make a Presentation Layer that

only supports communication with an Application Layer that does all the interpretation from the lower layers for it.

- **Second Decision Pertaining to Architecture Choice (In-memory Backup Data Structure):**
    - Identify And Describe an Alternative:
        - Backup Data File: A single file that represents the backup's contents, likely in JSON format.
    - Identify Pros Relative to Our Actual Choice:
        - It would make it so that we could recover from program crashes more easily, as long as the backup data file could be found, validated as correct for the selected iPhone backup, and reloaded.
        - It would not use up RAM.
    - Identify Cons Relative to Our Actual Choice:
        - It would take up disk space.
        - It would be slower to create and read from.
        - It would be possible for another program (or the user) to ruin the backup data file during the runtime of the program.

Software Design (What packages, classes, or other units of abstraction form these components, and what are each component's responsibilities?):

- **Components:**
    - CLI/TUI:
        - Units of Abstraction:
            - main_menu.py: Implements the Standard CLI, providing the interactive text-based menu system through which users load backups, configure settings, and initiate exports.
            - textual_main_menu.py: Implements the Textual Dashboard, a TUI alternative to the Standard CLI with equivalent menu-driven functionality.
            - Application Layer Services (services.py): Four service classes (BackupService, SettingsService, ConversionService, and ExportService) that bridge the Presentation Layer with the lower layers, managing backup state, export configuration, and extraction orchestration.

- Backup Locator & Validator:
  - Units of Abstraction:
    - get_device_info.py: Parses Info.plist in the backup root to extract device name, model identifier, iOS version, GUID, and backup date.
    - get_device_manifest.py: get_encryption_status() Reads Manifest.plist to determine if the backup is encrypted, returning an error immediately if so.
    - backup_model_builder.py: build_backup_model: Application Layer coordinator that calls all data readers, assembles the results into a BackupModel via album_builder and asset_builder, and returns a BackupModelResult.
- SQLite Query Component:
  - Units of Abstraction:
    - sqllite_connection_manager.py: Opens and closes SQLite database files safely.
    - sql_executor.py: Executes SQL queries provided by the Application Layer and returns raw result rows. Works alongside schema_inspector, which dynamically discovers the join table name and column layout linking albums to assets, accommodating schema variations across iOS versions.
    - row_mapper.py: Converts raw SQLite rows into dictionaries. album_reader and asset_reader execute the specific queries to retrieve raw data, while album_builder and asset_builder transform those rows into typed Album and Asset Pydantic objects with resolved relationships and smart folder memberships.
    - manifest_db_reader.py: Queries Manifest.db to locate the hashed path to Photos.sqlite within the backup directory.
    - asset_reader.py: Queries Photos.sqlite for asset data. Returns all assets from ZASSET joined with ZADDITIONALASSETATTRIBUTES. Returns all asset-to-album mappings from the join table. Queries

Manifest.db for the hashed fileID of a given relative path.
- album_reader.py: Queries Photos.sqlite for album data. Returns user-created and burst albums from 'ZGENERICALBUM'.

- File Extraction Engine:
  - Units of Abstraction:
    - extract_files.py: run_extraction_engine: The main entry point that orchestrates the full extraction loop, iterating over all assets and placing each into its appropriate collection folder(s) at the output root.
    - extraction_helpers.py: Resolves the active collections for each asset (honoring the configured Blacklist), determines the destination filename, and calls the Conversion Engine when a format conversion is needed.
    - collection_management.py: Deduplicates assets, separates burst frames from the main asset list, and builds the album UUID-to-title lookup map used throughout the extraction.
    - file_management.py: Handles all file system operations — creating destination folders, copying files, placing symlinks for non-exclusive assets, and sanitizing folder and file names.
- Conversion Engine:
  - Units of Abstraction:
    - convert_file.py: convert_asset: The main entry point for all conversions. Accepts an AssetToConvert object and routes it to the appropriate converter based on the file extension.
    - media_converter.py: convert_image: Converts HEIC images to JPG format using Pillow with the pillow-heif plugin.
    - media_converter.py: convert_video: Converts MOV videos to MP4 format using ffmpeg via imageio-ffmpeg. Live photo video components are handled as standard MOV files during this process.

Coding Guidelines (Briefly state why you chose these guidelines and how you plan to enforce them.):

- **Standard Python PEP 8 (https://peps.python.org/pep-0008/):**
  - We will be using the standard PEP 8 Python Style Guide for our project. To ensure that it is enforced, we will combine the use of a linter and the Black auto formatter to reduce time spent on correcting formatting issues.
- **SQL: GitLab Style Guide (SQL Style Guide):**
  - We will be using the GitLab Handbook SQL Style Guide for our project, along with the SQLFluff linter/auto formatter to fix any inconsistencies and easily enforce conventions.

Product Description:

- **Abstract:**
  - iExtract is a tool that helps people intelligently extract their photos and videos from their iPhones via reconstructing a folder structure and converting proprietary formats to common formats. This utility is dedicated to making iOS Photos app albums and collections easily convertible into folders en masse, which is not currently possible via any other tool than iMazing and a few other niche, monolithic tools. Additionally, iMazing has an expensive licensing model, as it provides many more features than just photo/video album to folder conversion, yet in a way that is not as focused.
- **Goal:**
  - The goal of this application is to solve the user experience issue regarding the either tedious, expensive, or iCloud-centric method of photo album and collection to local storage transfer that is available to iOS users at the moment. This app will facilitate a completely local transfer of albums and collections to local storage (converted into a folder-based format) on a user's PC, given they have created an unencrypted backup of their device, including their camera roll items.
- **Current Practice (Local Transfer Options):**
  - Windows Transfer Tools: Use the Windows Photos app (or older importer via File Explorer) to import your photos and videos without album/collection structure. Live photos get flattened via the Photos app. The older importer tool has the ability to organize photos and

videos based on time, and it does preserve live photos, but the key photo and video of each live photo will not necessarily be kept together.
    - iMazing: The camera roll related features of iMazing closest resemble what iExtract is aiming to accomplish, but the issue with iMazing is that since it includes so many more features in addition to those ones, it has a high price tag and is subscription-based. This business model is not ideal for users that simply wish to convert their albums and collections into folders without having to manually reconstruct them from a raw dump, which is the kind of import method that the Windows Photos app provides for free.
    - iTunes (Manually Copying User Created Folders of Photos/Videos from the iOS Files App): This is a free method that anyone can use. However, the issues with this method are that it is entirely manual, tedious, wastes storage space on your phone during the process, and is prone to conversion failures, especially regarding live photos, which get converted into still images if not manually converted to videos before exporting to the Files App. Additionally, the album/collection structure must be entirely manually recreated by the user in order to preserve it, which is extremely tedious and infeasible for large numbers of albums, especially since in order to copy an album to your Windows computer, for example, you must first create a folder of the same name in the iOS Files App, then you must ensure you convert all live photos to videos and re-add them to the original album, and then you must copy all of the items from said album into the folder of the same name in the Files App. After that, you must use iTunes to access the folder in which you stored these items and copy its contents to your computer.
- **Novelty:**
    - This application provides a cheaper and more focused method to users who want to avoid iCloud and backup their important albums and collections en masse, locally, in an automated manner. This application, unlike the alternatives, will not require an account, will be entirely free (or very cheap compared to the $29.99 per device price of iMazing), and will have power-user features, such as album blacklisting and whitelisting, ensuring users only back up the photos they want. It will be functional cross-platform, as it will use

Python, which will allow it to be used on Windows, Linux, and macOS, or anywhere that has a Python interpreter for the version we decide to use. Additionally, unlike some tools (iTunes) which only allow users to back up their items via a tedious process if their phone is still functional, this application will work on unencrypted/decrypted iOS backups, not active phones, which allows for much more flexibility in terms of use-cases.

- **Effects:**
    - If we are successful in creating this app in the given timeframe, there will finally be a free (or very inexpensive) tool that may be used by anyone to locally backup their iOS devices' photos and videos without the friction that currently exists, such as cost and tedium.
- **Use Cases (Functional Requirements):**
    1. User extracts a single collection (*Part of Feature 3.*):
        - Actors: Any iPhone user
        - Triggers: The user wants to extract a specific album or collection to a folder of the same name.
        - Preconditions:
            1. The user must have an album already saved in their iPhone backup that they want to convert into a folder.
            2. The user's computer must have enough storage to store the extracted items in.
        - Postconditions (success scenario): The selected album has been successfully exported to the folder of the same name.
        - List of steps (success scenario):
            1. The user opens the CLI app.
            2. The user selects "Export Specific Camera Roll Media" from the backup menu, then selects the target collection from the displayed list.
            3. The files in the album are successfully located (converted if needed or selected) and exported into the selected folder.
            4. The program displays a success message indicating that the same number of files as the user selected have been exported.
        - Extensions/variations of the success scenario

        1. The user has format conversions enabled in Settings (HEIC → JPG and/or MOV → MP4).
            - The files in the collection are successfully located, converted to the specified formats, and exported to the selected folder.
            - The program displays a completion message when finished.
        2. The files in the collection are successfully located, converted in the specified formats, and exported into the selected folder.
        3. The program displays a success message indicating that the same number of files as the user selected have been exported.
    - Exceptions: failure conditions and scenarios
        1. The user selects a collection for export, but the backup fails to load or is invalid:
            - The program displays an error indicating that the backup could not be loaded and returns to the main menu. The export is not initiated.
        2. When exporting the album, the parent folder for the new folder to be stored within, the destination folder, does not exist:
            - The program creates the non-existent folder and tells the user that the folder did not exist, so the program created it.
        3. During export, a file within the backup cannot be read:
            - The program skips the unreadable file, logs the error, and continues with the remaining assets.
2. User extracts all collections (*Part of Feature 1 or 2.*):
    - Actors: Any iPhone user
    - Triggers: The user wants to extract every album and collection from their backup at once.
    - Preconditions:
        1. The user must have an album already saved in their iPhone backup that they want to convert into a folder.
        2. The user's computer must have enough storage to store the extracted items in.

        3. The user's computer must have enough storage to store the extracted items.
    - Postconditions: All collections have been exported as folders to the chosen destination. For assets belonging to multiple collections, the real file is placed once and symlinks point to it from the remaining folders. If the OS does not support symlinks, the program duplicates the files instead.
    - List of Steps:
        1. The user opens iExtract, loads their backup, and selects "Export All Camera Roll Media" from the main menu.
        2. The user selects an output destination folder.
        3. The extraction engine creates a folder for each collection and copies the appropriate assets. For assets belonging to multiple collections, the real file is placed once and symlinks are used in the remaining folders. The program displays a completion message when finished.
    - Exceptions: If the OS does not support symlinks, the program copies the full files to every folder they belong in and alerts the user that symlinks were not created and files were duplicated instead.

3. User extracts a subset of collections by using the blacklist (or whitelist, which is just an inverted blacklist) (*Part of Feature 3.*):
    - Actors: An iPhone user
    - Triggers: The user wants to extract all collections except for a few specific ones they want to skip.
    - Preconditions: The user must have a loaded backup and enough free storage at the destination.
    - Postconditions: All collections except the blacklisted ones have been extracted as folders to the chosen destination.
    - List of Steps:
        1. The user opens iExtract and loads their backup.
        2. The user navigates to Settings and adds the collections they wish to skip to the Blacklist.
        3. The user selects "Export All Camera Roll Media" from the main menu and chooses a destination folder.

4. The extraction engine runs, skipping blacklisted collections and exporting all others.
    - The program displays a completion message when finished.
5. Exceptions: If a blacklisted collection name does not match any collection in the loaded backup, the program ignores the entry and proceeds with the export.

4. Perform any extraction with format conversions enabled in Settings (*Part of Feature 4.*):

- Actors: Any iPhone user
- Triggers: The user wants their extracted files to be in universally compatible formats (JPG or MP4) rather than Apple proprietary formats (HEIC or MOV).
- Preconditions: The user has a loaded backup and has enabled one or both conversions (HEIC → JPG, MOV → MP4) in the Conversion Settings menu before initiating an export.
- Postconditions: Applicable files in the extraction are converted to the target format and saved at the destination. Files whose format is not subject to any enabled conversion are copied as-is.
- List of Steps
    1. The user opens iExtract and loads their backup.
    2. The user navigates to Settings → Conversion Settings and enables HEIC → JPG and/or MOV → MP4.
    3. The user initiates any extraction (single collection, all collections, or blacklisted subset).
    4. During extraction, the Conversion Engine converts each eligible file to the target format before it is written to the destination folder.
    5. The program displays a completion message when the extraction finishes.
- Extensions/Variations: The user may combine conversion settings with the blacklist feature, applying conversions only to the non-blacklisted subset.
- Exceptions: If a file conversion fails (e.g., a corrupted HEIC), the program skips that file, logs the error, and continues with the remaining assets.

- **Non-functional Requirements:**
  - Portability:
    - iExtract must work without a separate codebase for each platform on any machine that:
      1. Meets the system requirements, in terms of hardware.
      2. Has the ability to run programs of the Python version we choose to use (*currently: Python 3.13*).
  - Usability:
    - The error messages produced by the CLI must be clear and understandable to non-technical users.
    - The CLI must display a summary of the exported contents once an operation is complete.
  - Maintainability:
    - The architecture of the app will be modular such that different features are part of distinctly separate components, enabling features to be developed and/or fixed quickly without the risk of breaking other features.
- **External Requirements:**
  - Understanding the iPhone backup format:
    - This application can only function successfully if it is able to successfully parse an iPhone backup, which is external and unalterable to the developers of this app. Thus, the requirement that we study the backup structure of iPhone backups is tantamount to the success of this project.
  - Understanding the user's filesystem:
    - Operating system rules across Windows, Linux, and macOS must be followed in order to enable backups to be found and parsed into extracted folders.
  - Users expectations:
    - Users expect instructions and error messages to be clearly written and resolvable without our direct support.
  - The project must be open-source and buildable:
    - The source code for this application must be publicly available for others to download, enabling them to run it directly, without having to download a package or installer.
  - Consistent offline approach:

        - The tool must work offline, as it is made to perform pure offline iPhone backup extractions.
- **Technical Approach:**
    - We are planning to use Python and libraries that work across Windows, Linux, and macOS in order to create this app in such a way that it is functional across all the major desktop platforms. The main user experience will be a command line interface, but in order to reduce friction for less technical users, we will include a detailed list of commands that will be shown on-screen once the user enters the "Help" section of the menu (*this feature has not yet been implemented*). An optional Textual TUI (Textual Dashboard) has also been implemented as a beta alternative to the Standard CLI. Additionally, we will be utilizing SQLite in our program to query the databases iOS uses to track which albums and collections media belongs to.
- **Timeline:**
    - Week 3:
        - Ensure the architecture of the app, including its design, in terms of how the directories and modularity will be implemented, is finished, and the CLI minimally works without errors (no major features must be implemented at this point).
    - Week 4:
        - Ensure that the CLI design is complete, in terms of the basic mockup, and ensure that the architecture has been decided so development can truly begin in full.
    - Week 5:
        - Ensure that the user can navigate a mockup of the final CLI and choose certain actions, with the first functional one being to "load a backup" successfully.
        - Begin training the AI model that we will be using to fulfil the "stretch goal" feature: Add AI Descriptions of Content to Photo Metadata.
    - Week 6:
        - Ensure that the BackupModel exists in its *final form* so that development can move forward on the components that rely upon it.

        - Ensure the Domain Layer components of the Backup Locator and Validator, Conversion Engine, and the SQL Command Facilitator have been prototyped.
        - Start work on the GUI prototype (using Textual) that utilizes textual text user interface (TUI) technology, which works upon the CLI, actually works to a minimum degree.
        - Train the AI model that we will be using to fulfil the "stretch goal" feature: Add AI Descriptions of Content to Photo Metadata.
    - Week 7:
        - Ensure that the BackupModel can be populated with device metadata, including basic details (no asset or album information yet) like the encryption status, phone model and submodel, backup GUID, etc.
        - Continue working on training the AI model that we will be using to fulfil the "stretch goal" feature: Add AI Descriptions of Content to Photo Metadata.
        - Continue working on the Textual GUI prototype.
        - Work on creating tests for the testing suite to increase robustness.
    - Week 8:
        - Ensure that the Conversion Engine passes tests to convert a given Asset from one format to another, covering MOV and HEIC (including live photo pairings) to common formats such as MP4 and JPEG at a user-specified quality level.
        - Ensure that, within the CLI, the blacklist and whitelist work to specify which albums and collections to include or exclude from extractions, and that non-exclusive assets shared across multiple collections are handled correctly via shortcuts or symlinks rather than duplication.
        - Continue working on the Textual GUI system.
        - Continue working on training the AI model that we will be using to fulfil the "stretch goal" feature: Add AI Descriptions of Content to Photo Metadata.
        - Continue to work on creating tests for the testing suite to increase robustness.
    - Week 9:

        - Continue training the AI descriptions feature for the beta showcase.
        - Continue with Textual GUI development.
        - Continue developing and testing iExtract's mass export feature (integrate blacklist and whitelist functionality).
        - Create the beta release, including the mass export feature as the first major feature for people to test.
        - Finish up the blacklist/whitelist functionality.
        - Finish up the individual collection export functionality.
        - Ensure that each and every known failure mode so far has a detailed error message associated with it that explains to a common user what they must do to mitigate the error in the future. Test the major features for bugs and usability issues and prepare for Week 10 *(1.0.0 Release Due Monday of Week 10)*.
    - Week 10+:
        - Implement some extra feature(s) (*those mentioned as Stretch Goals*) if we have time.

Team Process Description:

- **Software Toolset:**
    - Python: We are planning on using Python to develop our app quickly and incrementally. Additionally, using this language will allow us to deploy the app on multiple operating systems using one codebase if we don't utilize platform specific libraries (or if we implement multiple platform specific libraries to facilitate the same actions on the 3 OSes we are targeting: Windows 10+, Linux, and macOS.
    - SQLite: SQLite will need to be used for this application to function properly, as iPhone backups store a lot of data and metadata within SQLite databases such as Manifest.db and Photos.sqlite, which need to be parsed in order to reconstruct album and collection structure accurately in terms of folders. Moreover, Photos.sqlite contains metadata about images and videos, which we plan on preserving.
- **Risk Assessment:**
    - Scope Creep (Medium Likelihood, High Impact):
        - Evidence for Estimates: We have already come up with some advanced features, like SQL filtering or addons, and those

could make it infeasible to complete our project on time if we were to integrate them into the design from the beginning.
    - Steps to Reduce Likelihood & Impact & Permit Better Estimates: To avoid this issue, we will develop incrementally, adding basic features first, which will compose a minimum viable product before we move on to adding things like advanced filtering and detailed metadata preservation.
    - Plan for Detecting Problems: Checking in on the timeliness of the integration of one of the advanced features and ensuring that it does not introduce bugs to the core functionality of the program by running automated tests on the core components.
    - Mitigation Plan: If this is to occur, we will need to scale back the scope of our project to something achievable before the deadline that provides value to a user and is stable.
  - Team Members Not Communicating (Medium Likelihood, High Impact):
    - Evidence for Estimates: We have all been in teams in the past that have failed to communicate and caused some members to have to crunch to make up for the lack of communication and retain their own grades.
    - Steps to Reduce Likelihood & Impact & Permit Better Estimates: We will hold each other accountable if any of us are not responding, even to follow-up messages, making the completion of the project strained.
    - Plan for Detecting Problems: We will try to follow-up to any ignored messages, and if a teammate still isn't responding, we will ask another team member to do so as well, and if that doesn't work, we will talk to the unresponsive team member in person and ask about what's going on.
    - Mitigation Plan: Once we contact the unresponsive person in person, or if we can't, we will try to more thoroughly establish communication protocols amongst the team so we don't fall behind.
  - Difficulty of Understanding the iPhone Backups (Low Likelihood, High Impact):
    - Evidence for Estimates: Based on the manual exploration that we have done, we have been able to find albums, some

collections, and the items belonging to albums, as well as those items' metadata, partially, all in a repeatable pattern of steps. This makes it only slightly likely that it would be hard to understand the rest of what we need to understand and reconstruct a logical mapping of items to metadata.
    - Steps to Reduce Likelihood & Impact & Permit Better Estimates: We will ensure that we already understand how to perform the necessary operations on a backup that we are set out to automate manually before we begin attempting to automate those features.
    - Plan for Detecting Problems: To detect issues, we will test across backups of different iPhones and ensure that constructing a logical representation of the backup in memory works consistently.
    - Mitigation Plan: To counteract this risk, should it become reality, we will do research into the existing documentation around how iOS stores camera roll files and their metadata, which is publicly available.
  - Complexity of Keeping Architecture Pure (Medium Likelihood, Medium Impact):
    - Evidence for Estimates: It has already been a bit difficult to determine which architecture pattern we would like to use based on what our software will need to be doing, as it is not something that really fits nicely into one specific box.
    - Steps to Reduce Likelihood & Impact & Permit Better Estimates: We will attempt to refine the architecture of the app, which is the Layered Pattern, and ensure that it is consistent enough to where we don't have any major contradictions in our code that cause problems down the line.
    - Plan for Detecting Problems: To detect problems, we will go through the implementation of the layers of the pattern and see if anything within each layer violates a rule or not.
    - Mitigation Plan: To mitigate this risk, should it occur, we will move the incorrectly implemented functionality to the correct layer, so that we don't build up technical debt.
  - Inability to Follow Established Conventions (Low Likelihood, Low Impact):

        - Evidence for Estimates: We all have different coding styles, which we have already noticed based on the code that we have written in in-class exercises, and based on the reactions to the code that one team member wrote for the CLI mockup.
        - Steps to Reduce Likelihood & Impact & Permit Better Estimates: We will each read the style guides for the Python and SQL that we have put in this document, limiting the time it will take to fix any issues.
        - Plan for Detecting Problems: We will use a linter on each file to detect code inconsistencies.
        - Mitigation Plan: We will use linters and code auto-formatters to fix any issues that remain after manual fixing, and fix any unresolved issues after that manually.
- **How Risk Assessment Has Changed:**
    - Our risk assessment has become more precise and based on the project's technical specifications established since the Requirements document was made. In the beginning, our risks were wide-ranging and based on presumptions regarding the iOS backup format and the intricacy of the extraction process. Now, our risk estimates are more detailed and, the list itself, more complete. We have more extensive plans for problem prevention, along with plans to correct issues, may they still occur.

INSTRUCTIONS TO COMPLETE PROJECT SCHEDULE:

- Identify milestones (external and internal), define tasks along with effort estimates (at granularity no coarser than 1-person-week units), and identify dependencies among them. (What has to be complete before you can begin implementing component X? What has to be complete before you can start testing component X? What has to be complete before you can run an entire (small) use case?) This should reflect your actual plan of work, possibly including items your team has already completed.
- To build a schedule, start with your major milestones (tend to be noun-like) and fill in the tasks (tend to start with a verb) that will allow you to achieve them. A simple table is sufficient for this size of a project.
- **Project Schedule:**

ID	Milestone / Task	Role	Effort Estimate	Dependencies
M1	Internal: System Architecture & Skeleton [Week 3]	--	--	--
T1.1	Create skeleton code for all components with modular layered design	Developer	1 week	None
T1.2	Create mock data for initial feature testing	Director	1 day	T1.1
T1.3	Host weekly meeting & confirm skeleton functionality	Director	1 day	T1.1
M2	External: UI/UX Design Approval [Week 4]	--	--	--
T2.1	Identify user flow paths for core features	UI/UX & Director	3 days	M1
T2.2	Create initial CLI design and mockup	UI/UX & Director	1 week	T2.1
M3	Internal: BackupModel Fully Populated [Week 6–7]	--	--	--

T3.1	Define and finalize BackupModel domain layer (all data structures)	Developer & Director	3 days	T1.1
T3.2	Implement Backup Locator & Validator (device metadata from Info.plist / Manifest.plist)	Developer & Director	3 days	T3.1
T3.3	Implement SQL Command Facilitator (Photos.sqlite & Manifest.db queries, asset/album population)	Developer & Director	1 week	T3.2
T3.4	Write unit tests for BackupModel builder and SQL component	QA & Developer	3 days	T3.2, T3.3
M4	Internal: Conversion Engine Complete [Week 8]	--	--	--
T4.1	Implement ImageConverter (HEIC → JPEG/PNG at user-specified quality)	Developer	3 days	T3.1
T4.2	Implement VideoConverter (MOV → MP4 at user-specified quality)	Developer	3 days	T3.1

T4.3	Implement LivePhotoConverter (Reuse the Other Conversion Components) (HEIC + MOV → MP4)	QA & Director	3 days	T4.1, T4.2
T4.4	Write unit tests for all conversion paths	QA	3 days	T4.1, T4.2, T4.3
M5	Internal: Core Functional Readiness (Features 1 & 2 — Mass Export) [Week 8–9]	--	--	--
T5.1	Implement ExtractionPlanner (determine what to extract based on BackupModel)	Developer & Director	3 days	T3.3
T5.2	Implement FolderBuilder (create destination folder structure mirroring album hierarchy)	Developer & Director	2 days	T5.1
T5.3	Implement FileCopier (copy media files, invoke Conversion Engine when needed)	Developer & Director	3 days	T5.2, T4.1, T4.2
T5.4	Implement SymlinkManager (handle non-exclusive assets across multiple albums)	Developer & Director	2 days	T5.3

- **Team**

T5.5	Integrate File Extraction Engine backend with CLI	UI/UX & Developer	3 days	T2.2, T5.4

Structure (Member Roles & Justifications):

- Kevin (Director): This project needs a director so that the direction of the project stays on track and within scope. This role will prevent us from wasting time or not knowing what to do while waiting for others to finish their work. Kevin is being chosen for this role because the project was his idea, so it makes sense that the general direction of it should stay in line with that idea.
- Noah (UI/UX): A good program requires intuitive controls and a navigable menu. We want our program to be user-friendly and easy to use. Noah is being chosen for this role because he has prior experience in creating UI and designing websites.
- Brendon (QA Tester): Having a QA tester ensures that we catch functional failures and edge-cases. The person taking on this role will need to create test cases and give us criteria for correctness. He will also spend time to validate that the product is usable and reliable for users.
- Sam (Developer): This program requires developers to ensure that the project is functional and meets the product description. This role will work on the back end of the program to build up all the required features and will be continuously reviewing the code to ensure that the quality is upheld throughout the codebase. Sam is being chosen for this role because he has prior experience in working as a back end developer in previous projects.

INSTRUCTIONS TO COMPLETE TEST PLAN: Describe what aspects of your system you plan to test and why they are sufficient, as well as how specifically you plan to test those aspects in a disciplined way. Describe a strategy for each of unit testing, system (integration) testing, and usability testing, along with any specific test suites identified to capture the requirements.
NOTE: We require that you use GitHub Issues to track bugs that occur during use and testing.

- **Test Plan:**

- We will test our CLI menu to make sure that each option works accordingly. We will also independently test each feature in our app and also have a final test of the whole app when we complete the beta version. We plan to conduct manual tests at the end of each week so we can catch and fix any bugs that we find in the following week. We will be tracking all of our tests from our regression testing sheet and make any updates accordingly. We will also have acceptance criterias for each feature to make sure that we meet all of our functional and non-functional requirements. We will also track each individual issue as it appears using GitHub issues.
- Unit Testing
  - We will utilize unit testing for the Application and Data Layers. More specifically, targeting the logic functionalities like validation logic, data transformation, and decision making algorithms. We will utilize Python's built-in 'unittest' framework alongside unittest.mock for mocking, to isolate the Application and Data Layers. We chose unittest over alternatives because it is included in Python's standard library, requiring no additional dependencies, and integrates directly with our CI pipeline via a single 'python -m unittest discover tests' command.
  - We considered but decided against the following alternatives: 'Pytest', which offers more expressive syntax and better output formatting but introduces an external dependency and was more than we needed for this project; 'Nose2', which extends 'unittest' but is largely in maintenance mode and has a smaller community; and 'Hypothesis', which provides property-based testing and would be useful for fuzzing our SQL query logic, but was too complex to justify for our current test coverage goals.
- Test Automation Infrastructure and Continuous Integration
  - Our CI service is GitHub Actions, linked directly to our repository at https://github.com/Gustakev/cs362-class-project. GitHub Actions is configured via a YAML workflow file in '.github/workflows/' that triggers automatically on every push and pull request to the main branch, running 'python -m unittest discover tests' against the commit. We chose GitHub

Actions because it is natively integrated into GitHub with no external account or service required, is free for public repositories, and requires minimal configuration to get going for a Python project. The following table compares the CI services we considered and what we decided on:

CI Service Option	Pros	Cons
GitHub Actions	Native GitHub integration, no external setup, free for public repositories, large marketplace of reusable actions, YAML config file lives in the repo	Less flexible than dedicated CI tools for complex pipelines, limited free minutes for private repos (but ours is public)
CircleCI	Highly configurable, fast build times, good parallelism support, free tier available	Requires external account and service linkage, more complex setup than GitHub Actions for a simple Python project
Azure Pipelines	Enterprise-grade, great Windows support, free for open-source usage	Overkill for a small academic project, more complex UI and configuration, primarily designed for Microsoft ecosystem workflows, which we aren't really using
Travis CI	Simple YAML configuration file, long-established community, good Python support	Free tier was significantly reduced in 2021 and is now very limited for open-source projects, making it less practical for teams if the team isn't willing to pay for a paid plan

- Usability Testing
  - We will conduct observational tests, where participants will attempt to perform core functionalities. This will help validate that the CLI is usable and conveys understandable messages.
- This test strategy is sufficient because it covers the entire system at three different levels. Unit testing ensures that the (Data Layer)

complex logic is bug free in isolation. Integration testing makes sure that file system operations (Application Layer) work on the host machine. Usability testing makes sure that the CLI (Presentation Layer) is navigable and intuitive.

- **Documentation Plan:**
    - Readme: Include instructions on how to use the app within the file 'README.md' hosted on the GitHub repository. We will update these instructions, starting in Week 6, and until the release of the project.
    - Help Section: The main menu of our app will include a help section which, when entered, will display all the different sorts of clarifications and FAQ answers to common questions and confusions regarding the functionality of the app.
- **External Feedback:**
    - Extract All Collections (*Feature Complete*):
        - At this point, we will need to ensure that the export actually exports everything, not including thumbnails and other files that are meant to be temporary and aren't shown in the Photos app on iOS. To do so, we may recruit some people we know in order to test this tool on backups of their phones, as well as to test this feature in front of TAs and/or the professor to ensure that the code is not only functional, but that it is also efficient and complies with common standards.
    - Extract Only Certain Collections (*Feature Complete*):
        - At this point, we will need to ensure that when a user uses the application with the intention of exporting only certain albums and collections, only those are included in the export, and that all the real items are included in each exported album and collection, without the temporary files and thumbnails. Once again, to do this, we will need to have multiple people, some that we know in our personal lives, and perhaps some TAs, attempt to use the tool for this purpose and demonstrate its usage on backups of their devices. We will also need to have TAs and other people with expertise look over our code and help us catch bad practices early on, so that we don't build up technical debt.
    - Conversion Quality Testing (*Feature Complete*):

- When a user chooses that they would like to convert proprietary file types like live photos into common file types, those conversions need to be done properly in every case. In order to ensure that this is true, we need to test the program on multiple backups from different devices, ensuring consistent behavior. We will likely need external sources to volunteer backups for us to perform operations on, or we will need to create more backups to examine ourselves. Alternatively, we could have people try out the tool on their own time, so that we don't need to invade their privacy, and they could tell us if the conversion process was successful or not, allowing us to get real world "customer" feedback.

Major Features (4+):

1. Mass Album -> Folder Conversion
2. Mass Collection -> Folder Conversion
3. Folder & Collection Blacklisting (& Whitelisting) System
    a. This feature will enable users to choose which albums and collections they want to convert into folders.
4. Proprietary File Type -> Common File Type Conversion Options

Stretch Goals (2+):

1. Extract Folders of Files from iOS Files App & Incorporate Blacklist (& Whitelist) Features to This Feature
2. Custom Folder Creation Implemented Via SQL Filtering
    - The user would use a UI (the CLI or TUI) to apply the filters, not needing to enter the commands themselves.
3. Implement a GUI (Textual UI) Alternative to the CLI — Completed
4. Add AI Generated Descriptions of Extraction Folder Photo Content to App — Partially Complete (Integrated as Beta Demo)

Notes:

- The major features should constitute a minimal viable product (MVP).

Project Revision From Feedback:

- Handling API Server Errors

One feedback pointed out that the system could encounter a SELEOFError when the API server becomes overloaded. To address this issue, we updated the code to include a try and except block to handle this exception. This allows the program to catch the error instead of crashing and ensures the system can respond more gracefully when the server is under heavy load.

- Improving AI Caption Accuracy

  Another piece of feedback mentioned that some of the AI-generated descriptions were incorrect. After reviewing the issue, we determined that the beta version of the model had not been trained for enough epochs. To improve the model's performance, I trained the model with additional epochs so that it could better learn the dataset and generate more accurate captions.

- Improving Documentation

  Feedback also indicated that the README.md file needed improvement. Specifically, the documentation could be expanded by adding a dedicated Documentation section with helpful links. Additionally, some parts of the Table of Contents were not functioning correctly. I updated the README by fixing the broken links and adding a documentation link section to improve usability and navigation for users.

Project Milestone-7: Reflections:

1. **Sam Daughtry:**
   a. What I Learned:
      i. I
      ii. I
      iii. I
   b. What went well:
      O
   c. What would I do differently?:
      O

1. **Noah Gregie:**
   a. What I Learned:

i. The main thing I learnt from this project was the process of turning an idea into a functioning software. Most projects I've worked on have clear requirements and goals, but this one left it up to us. This really emphasized the importance of the requirements phase. As I know from experience, working towards an unclear goal is very difficult.
ii. Another important lesson was learning how to work with the specific architecture we chose, or the separation of work in general. Before this project, I had never worked on a UI/ interface while other people worked on the backend. This meant that at the beginning, I didn't want to create menus that would force the backend team to create the backend based on the menus I made. However, with good communication, this ended up not being a big deal.
iii. The final lesson that I already knew but was reinforced greatly in this project was the idea of continuous development. There was not a single moment during this project when I thought that I couldn't add or change anything. Of course, the main features have been implemented, but every time I go back, I always find something that could be optimized or changed. This also showed me why test suites are so important. When you are constantly making small changes, at the end of the day, you still need the main software to work.

b. What went well:
I think the main factor behind the project's success was our clear vision. There was a clearly defined goal that we could all understand and work towards. We might have had different approaches or ideas, but we were all working towards the same thing. In my opinion, it's much better to differ on implementation than on what purpose the software should serve.

c. What would I do differently?:
The one thing that I wish I had done differently was to work more on the backend. I did find working on the interface to be challenging and enjoyable, but it is something that I have done before. Usually it's the backend of the software that makes it unique, and working more on that is definitely what I would do differently next time.

1. **Kevin Gustafson:**
    a. What I Learned:
        i. By working on this project, I learned more about software architectures, which will help me professionally in the future. We decided to go with Layered Architecture for this project, which was sort of hard to plan, but the fact that we decided to go with it was ultimately beneficial, as it allowed us all to work on independent components at the same time without breaking things (which may have happened if we used a different, more monolithic architecture).
        ii. I also learned how to delineate responsibilities across a team, as I was the designated Director of the project (who also did a lot of various development, but mostly backend stuff). I learned that keeping the team up to date with goals for the week was something I needed to work on pretty early on. I ended up getting into the habit of communicating frequently, which helped me, in terms of telling the team what needed to be worked on and keeping the reports and GitHub issues up-to-date.
        iii. Additionally, I learned that making continuous integration tests is important, even if time consuming, so that old functionalities are not broken by new features or bug fixes. Using CI ensures that we know when something is broken and why, helping us fix it quicker.
    b. What went well:
    We were able to get all the major features (along with one stretch goal: AI Photo Descriptions) done. This is a definite win for the team, as we had struggled early on to get a good start on the code, with us having many conflicting schedules and ideas. Despite that, we were able to work out responsibilities and a schedule that allowed us to finish the project on time.
    c. What would I do differently?:
    If I could've, I would've only done one thing differently: I would have (in more depth) decided, with the team, the specifics of the project early on, making detailed pseudocode (because before I started to do that, there were times in which some teammates misinterpreted what I had asked them to do and developed something slightly

different that needed changes later). Doing so could have saved us some time in the long-run.

1. **Brendon Wong:**
    a. What I Learned:
        i. I learned a lot about software architecture. Before this class, I never had to build my software around a software architecture, but it taught me how to build a more organized software.
        ii. I learned the importance of communication within a team. Since multiple people are working on different parts of the system, it is important that everyone understands the overall design and stays informed about changes or updates. When communication was clear, it was easier to coordinate tasks and avoid duplicated work.
        iii. I learned the importance of clear requirements. During the project, we received feedback from the mid-term report that required us to revise parts of our application. This experience showed me how important it is to clearly define system requirements while still being flexible enough to adapt when improvements are needed. Our team had to update aspects of the app while still maintaining the original goals and requirements of the project.
    b. What went well:
    One thing that went well was how quickly the tasks were completed. Oftentimes, we would get the tasks completed very quickly because we had someone in our group allocating the work amongst the team members.
    c. What would I do differently?:
    One thing I would do differently is have better communication. There were times where I would get busy with my other classes and wasn't entirely focused on this class, so my communication wasn't the best.